innovate achieve lead

BITS Pilani

Pilani Campus

BITS Pilani

Pilani | Dubai | Goa | Hyderabad

By Prof A R Rahman

CSIS Group WILP

innovate achieve lead

BITS Pilani
Pilani Campus

Course Name: Introduction to DevOps
Course Code : CSI ZG514/SE ZG514

By Prof A R Rahman
CSIS Group WILP

CS - 8 Continuous Integration and Continuous Delivery

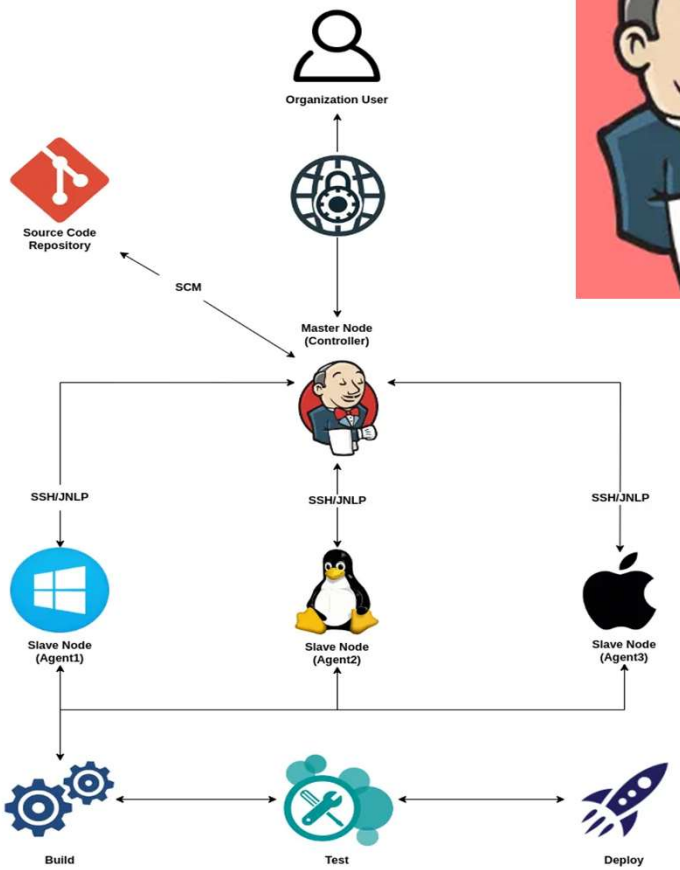
Coverage

Continuous Integration and Continuous Delivery	• Jenkins Architecture • Integrating Source code management, build, testing tools etc., with Jenkins - plugins • Artefacts management • Setting up the Continuous Integration pipeline • Continuous delivery to staging environment or the pre-production environment • Self-healing systems

Basic Jenkins Architecture

- Jenkins is a powerful open-source automation server used to build, test, and deploy software in a continuous integration/continuous deployment (CI/CD) pipeline.

- In a typical Jenkins setup, jobs are orchestrated by a master node and distributed to various agents, allowing for flexible builds across different environments.



Basic Jenkins Architecture

- 1. Organization User
  At the top of this architecture is the **Organization User**, the individual or team responsible for making changes to the codebase.
- These changes trigger the Jenkins build process.

- 2. Source Code Repository
  The source code is managed in a **Source Code Repository**, typically using a version control system (VCS) like Git.
- Jenkins is integrated with the repository through SCM (Source Code Management), allowing it to monitor changes (commits or merges).
- Once a change is detected, Jenkins pulls the latest version of the code to begin the build process.

Basic Jenkins Architecture

- 3. Master Node (Controller)
- The **Master Node**, also known as the **Controller**, is the brain of the Jenkins architecture.
- It schedules build jobs, monitors the agents (slave nodes), and assigns jobs based on predefined rules.
- The master node doesn't usually execute the build or test jobs but instead delegates them to agents that are better suited for those tasks.

Basic Jenkins Architecture

- 4. Slave Nodes (Agents)
- Jenkins can distribute workloads across various Slave Nodes (Agents).
- Each agent is a machine (physical or virtual) that performs the actual work.
- In the diagram above, we have three different types of agents:

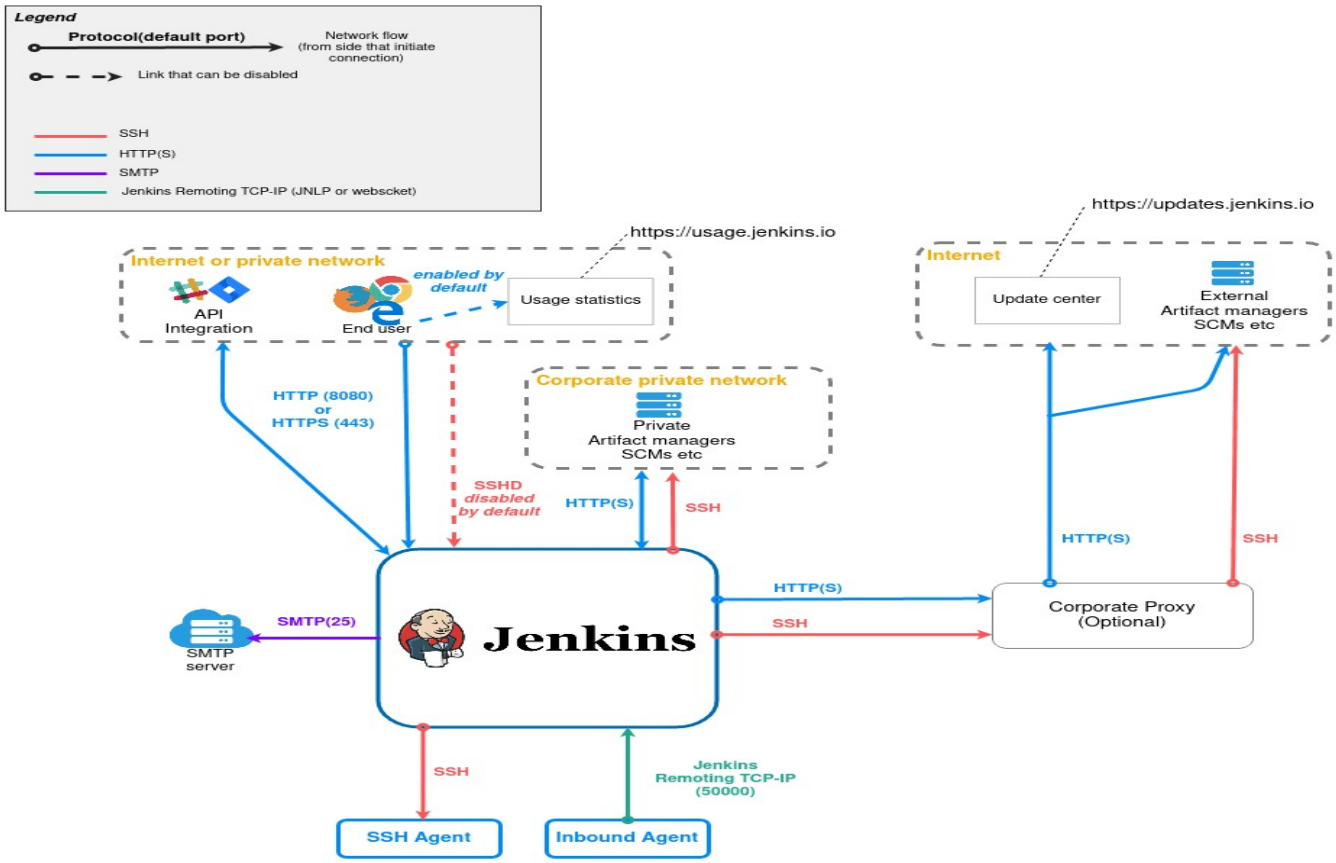
Agent1: A Windows-based machine that handles Windows-specific jobs.
Agent2: A Linux-based machine for running jobs in Linux environments.
Agent3: A MacOS machine, used when builds or tests require a Mac environment.

The communication between the master node and the agents is typically established via SSH or JNLP.

Basic Jenkins Architecture

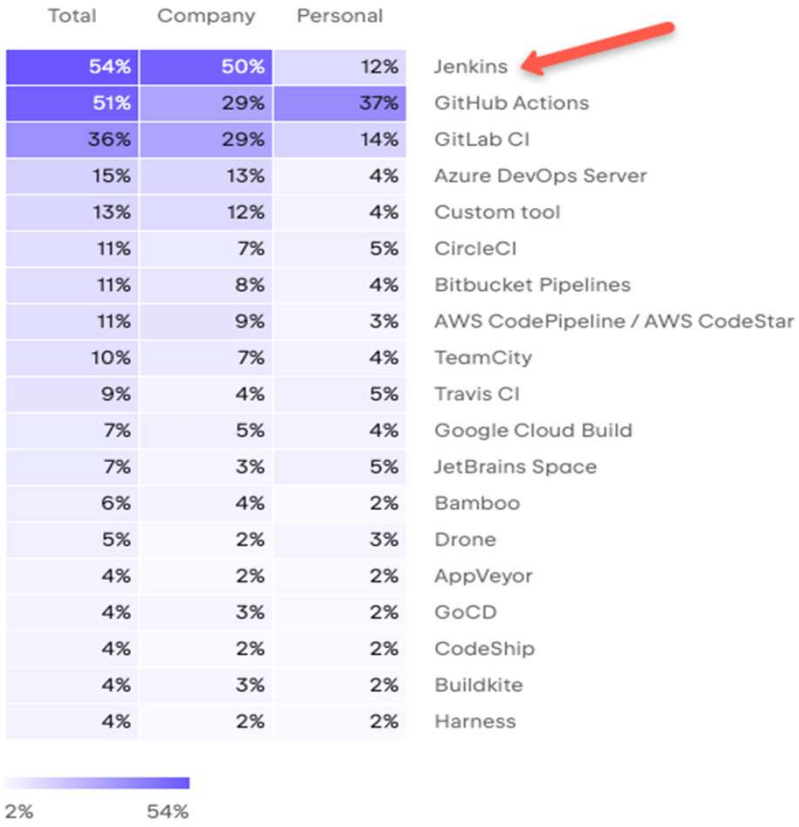
- 5. Build, Test, Deploy
- Once the agents receive jobs from the master, they proceed with the **Build**, **Test**, and **Deploy** stages:
- **Build**: The source code is compiled, and build artifacts are created.
- **Test**: Automated tests are run to ensure the build is stable and free of defects.
- **Deploy**: If the build and tests pass, the application can be deployed to staging or production environments.
- The ability to run jobs on different agents allows for greater flexibility and efficiency, especially in large projects with varying platform requirements.

High level overview of the Jenkins architecture.



High level overview of the Jenkins architecture.

- Jenkins has huge community support and an ocean of plugins that can integrate with many open-source and enterprise tools to make your life so easy.
- Despite the growing popularity of CI/CD tools like GitLab CI and GitHub Actions, which are increasingly being adopted by organizations for their CI/CD needs, Jenkins remains widely used in many organizations.
- As per the Developer Ecosystem Report, 54% of the developers use Jenkins for CI/CD.



Total	Company	Personal	
54%	50%	12%	Jenkins
51%	29%	37%	GitHub Actions
36%	29%	14%	GitLab CI
15%	13%	4%	Azure DevOps Server
13%	12%	4%	Custom tool
11%	7%	5%	CircleCI
11%	8%	4%	Bitbucket Pipelines
11%	9%	3%	AWS CodePipeline / AWS CodeStar
10%	7%	4%	TeamCity
9%	4%	5%	Travis CI
7%	5%	4%	Google Cloud Build
7%	3%	5%	JetBrains Space
6%	4%	2%	Bamboo
5%	2%	3%	Drone
4%	2%	2%	AppVeyor
4%	3%	2%	GoCD
4%	2%	2%	CodeShip
4%	3%	2%	Buildkite
4%	2%	2%	Harness

High level overview of the Jenkins architecture.

Jenkins is commonly used for the following.

1.Continuous Integration for application and infrastructure code.

2.Continuous delivery pipelines to deploy the application to different environments using Jenkins pipeline as code.

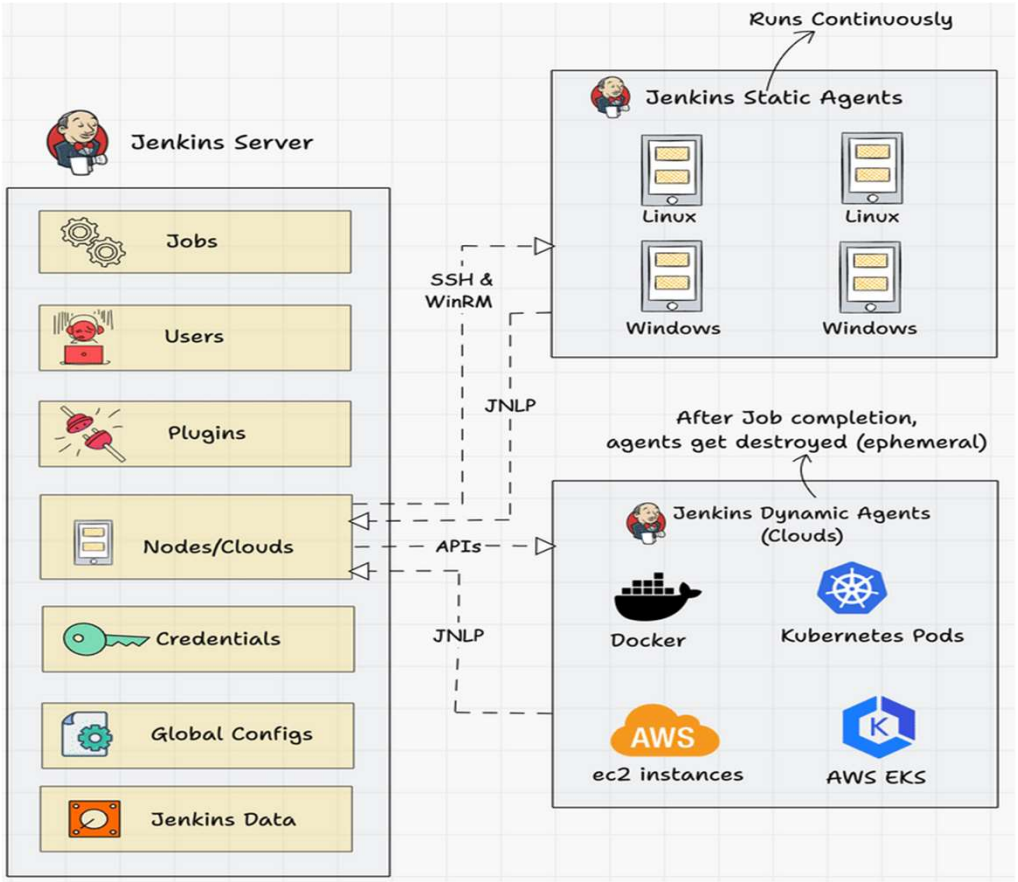
3.Infrastructure component deployment and management using IaC Tools.

4.Run batch operations using Jenkins jobs.

5.Run ad-hoc operations like backups, cleanups, remote script execution, event triggers, etc.

6.Jenkins Architecture

overall architecture of Jenkins and the connectivity workflow



overall architecture of Jenkins and the connectivity workflow

- Following are the key components in Jenkins
- Jenkins Master Node
- Jenkins Agent Nodes/Clouds
- Jenkins Web Interface

- Let's look at each component in detail.

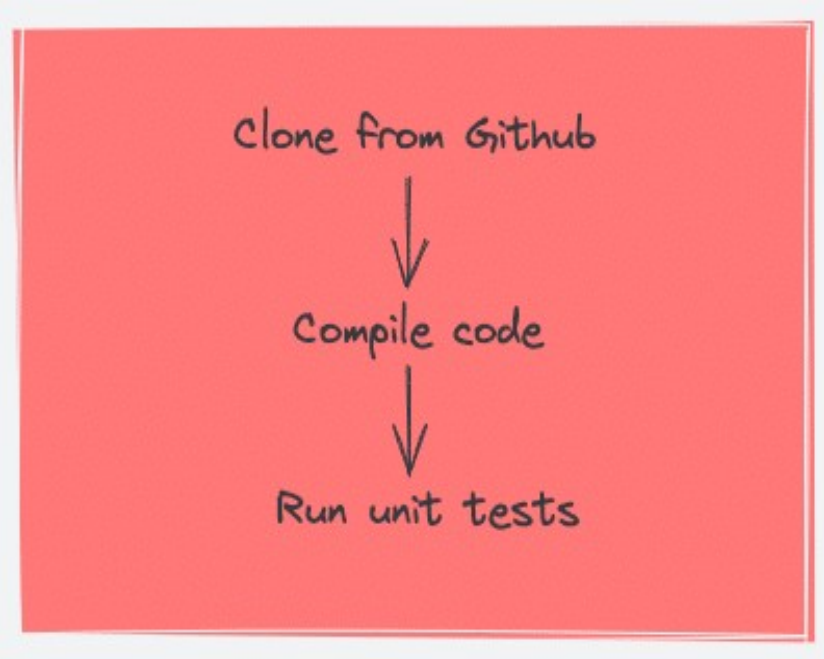
- Jenkins Server (Formerly Master)
- Jenkins's server or master node holds all key configurations. Jenkins master server is like a control server that orchestrates all the workflow defined in the pipelines.

- For example, scheduling a job, monitoring the jobs, etc.
- Let's have a look at the key Jenkins master components.

overall architecture of Jenkins and the connectivity workflow

- Jenkins Jobs
- A job is a collection of steps that you can use to build your source code, test your code, run a shell script, run an Ansible role in a remote host or execute a terraform play, etc.

- We normally call it a Jenkins pipeline.



overall architecture of Jenkins and the connectivity workflow

- If you translate the above steps to a Jenkins pipeline job, it looks like the following.

```
        stage('Code Checkout') {
            steps {
                checkout([
                    $class: 'GitSCM',
                    branches: [[name: '*/master']],
                    userRemoteConfigs: [[url: 'https://github.com/spring-
projects/spring-petclinic.git']]
                ])
            }
        }

        stage('Code Build') {
            steps {
                 sh 'mvn install -Dmaven.test.skip=true'
            }
        }
```

There are multiple job types available to support your workflow for continuous integration & continuous delivery.

Jenkins Plugins

- Plugins are official and community-developed modules that you can install on your Jenkins server.
- It helps you with more functionalities that are not natively available in Jenkins.
- For example, if you want to upload a file to s3 bucket from Jenkins, you can install an AWS Jenkins plugin and use the abstracted plugin functionalities to upload the file rather than writing your own logic in AWS CLI.
- The plugin takes care of error and exception handling.
- Here is an example, of s3 file upload functionality provided by the AWS Steps plugin

Jenkins Plugins

```
s3Upload(
    file:'file.txt',
    bucket:'my-bucket',
    path:'path/to/target/file.txt'
    )

s3Upload(
    file:'someFolder',
    bucket:'my-bucket',
    path:'path/to/targetFolder/'
    )
```

- You can install/upgrade all the available plugins from the Jenkins dashboard itself. For corporate network, you will have to setup a proxy details to connect to the plugin repository.
- You can also download the plugin file and install it by copying it to the plugins directory under /var/lib/jenkins folder.
- You can also develop your custom plugins. Check out all plugins from the Jenkins Plugin Index

Jenkins Global Security

- Jenkins has the following type of primary authentication methods.
- **Jenkins's own user database:-** Set of users maintained by Jenkins's own database.
- When we say database, its all-flat config files (XML files).
- **LDAP Integration**:- Jenkins authentication using corporate LDAP configuration.
- **SAML Single Sign On(SSO)**: Support single sign on using providers like Okta, AzureAD, Auth0 etc.
- With Jenkins matric-based security you can further assign roles to users on what permission they will have on Jenkins.

Jenkins Credentials

- When you set up Jenkins pipelines, there are scenarios where it needs to connect to a cloud account, a server, a database, or an API endpoint using secrets.

- In Jenkins, you can save different types of secrets as a credential.

- Secret text
- Username & password
- SSH keys
- All credentials are encrypted (AES) by Jenkins.

- The secrets are stored in $JENKINS_HOME/secrets/ directory.

- It is very important to secure this directory and exclude it from Jenkins backups.

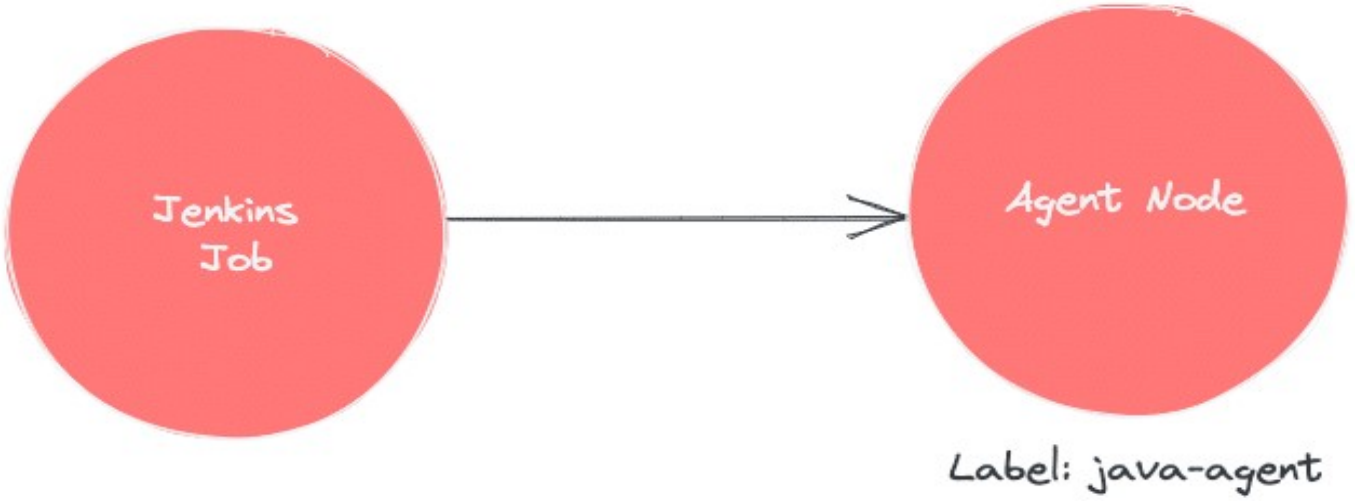
Jenkins Nodes/Clouds

- You can configure multiple agent nodes (Linux/Windows) or clouds (docker, Kubernetes) for executing Jenkins jobs.
- Jenkins Global Settings (Configure System)
- Under Jenkins global configuration, you have all the configurations of installed plugins and native Jenkins global configurations.
- Also, you can configure global environment variables under this section.
- For example, you can store the tools (Nexus, SonarQube, etc.) URLs as global environment variables and use them in the pipeline.
- This way it is easier to make URL changes that get reflected in all the Jenkins jobs.

Jenkins Nodes/Clouds

- Jenkins Logs
- Provides logging information on all Jenkins server actions including job logs, plugin logs, webhook logs, etc.
- Jenkins Agent
- Jenkins agents are the worker nodes that actually execute all the steps mentioned in a Job.
- When you create a Jenkins job, you have to assign an agent to it.
- Every agent has a label as a unique identifier.

Jenkins Nodes/Clouds



When you trigger a Jenkins job from the master, the actual execution happens on the agent node that is configured in the job.

Jenkins Nodes/Clouds

- You can have any number of Jenkins agents attached to a master with a combination of Windows, Linux servers, and even containers as build agents.
- Also, you can restrict jobs to run on specific agents, depending on the use case.
- For example, if you have an agent with java 8 configurations, you can assign this agent for jobs that require Java 8 environment.
- There is no single standard for using the agents.
- You can set up a workflow and strategy based on your project needs.

Jenkins Nodes/Clouds

- Jenkins server-agent Connectivity
- You can connect a Jenkins master and agent in two ways

- **Using the SSH method:** Uses the ssh protocol to connect to the agent.
- The connection gets initiated from the Jenkins master.
- Ther should be connectivity over port 22 between master and agent.

- **Using the JNLP method:** Uses java JNLP protocol (Java Network Launch Protocol).

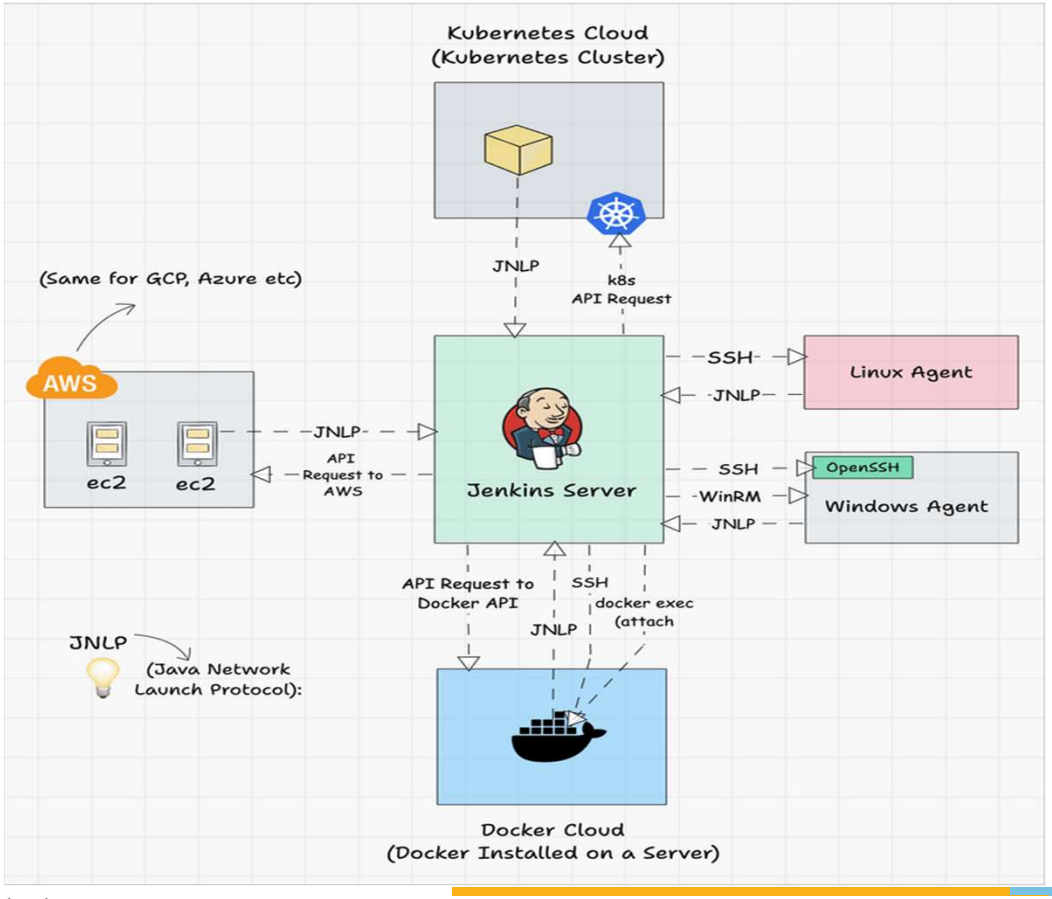
- In this method, a java agent gets initiated from the agent with Jenkins master details.
- For this, the master nodes firewall should allow connectivity on specified JNLP port.
- Typically, the port assigned will be 50000.
- This value is configurable.

Jenkins Nodes/Clouds

- There are two types of Jenkins agents
- **Agent Nodes:** These are servers (Windows/Linux) that will be configured as static agents.
- These agents will be up and running all the time and stay connected to the Jenkins server.
- Organizations use custom scripts to shut down and restart the agents when is not used.
- Typically, during nights & weekends.

- **Agent Clouds:** Jenkins Cloud agent is a concept of having dynamic agents.
- Means, whenever you trigger a job, a agent gets deployed as a VM/container on demand and gets deleted once the job is completed.
- This method saves money in terms of infra cost when you have a huge Jenkins ecosystem and continuous builds.

High-level view of different types of agents and connectivity types



You can refer to the tutorials to understand more about Jenkins clouds.

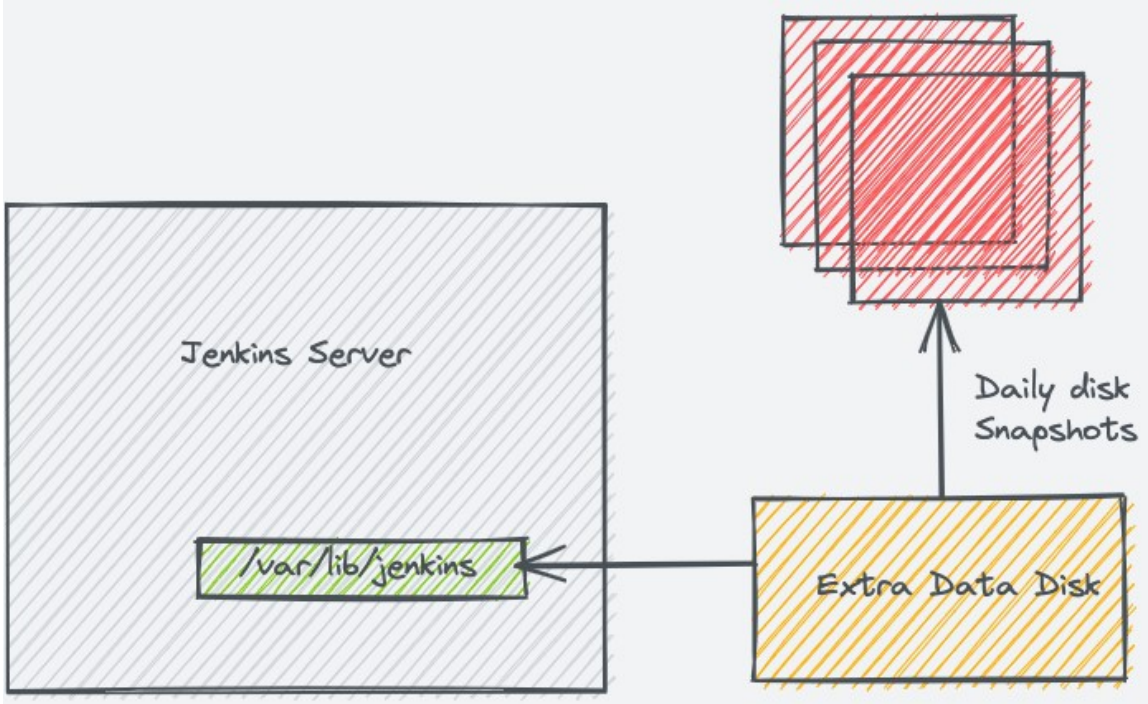
1.Docker container as build agents.

2.Setup Jenkins Build Agents on Kubernetes Pods

Jenkins Data

- All the Jenkins data gets stored in the following folder location.
- /var/lib/jenkins/
- Data includes all jobs config files, plugins configs, secrets, node information, etc.
- It makes Jenkins migration very easy as compared to other tools.
- If you take a look at /var/lib/jenkins/ you will find most of the configurations in xml format.
- It is essential to back up the Jenkins data folder every day.
- For some reason, if your Jenkins server data gets corrupt, you can restore the whole Jenkins with the data backup.

Jenkins Data



Ideally, when deploying Jenkins in production, a dedicated **extra volume is attached** to the Jenkins servers that hold all the Jenkins data.

Jenkins Data

- Jenkins is the ultimate tool for building, testing, and deploying your software.
- It provides several benefits, including automation, flexibility, and integrations with other tools and technologies.
- Jenkins is highly customizable, making it suitable for small-scale projects to large-scale enterprise projects.
- It is easy to install and set up, and creating and running your first Jenkins job is a straightforward process.
- Try Jenkins today and experience the power of automation in software development.

Integrating Source code management and builds with Jenkins

- **Step 1: Install Git Plugin in Jenkins**
- Open your Jenkins dashboard.
- Navigate to “Manage Jenkins” > “Manage Plugins.”
- Switch to the “Available” tab.
- Search for “Git Plugin” in the filter box.
- Check the box next to “Git Plugin” and click “Install without restart.”

Integrating Source code management and builds with Jenkins

- **Step 2: Configure Tools in Jenkins**

- For Jenkins on Linux, Git is already configured.

- For Jenkins on Windows, Git need to be configured:

- Go to “Manage Jenkins” > “Tools”
- Scroll down to the “Git” section.
- Click on “Add Git” to add Git installations.

- Enter a name (Default) for the Git installation and specify the path to the Git executable (C:\Program Files\Git\git.exe). Ensure Git is installed on the Jenkins server.

- Click “Save” to apply the changes.

Integrating Source code management and builds with Jenkins

- Step 3: Create a New Jenkins Job
- From the Jenkins dashboard, click on “New Item.”
- Enter a name for your job and select “Freestyle project.”
- Click “OK” to create the job.

Integrating Source code management and builds with Jenkins

- Step 4: Configure Source Code Management (Git) in Jenkins Job
- Select “Git” from the dropdown menu.
- Enter the repository URL of your Git repository.
- Optionally, specify the branch(es) you want to build.
- You can also configure credentials if your Git repository requires authentication.
- Click “Save” to apply the changes.

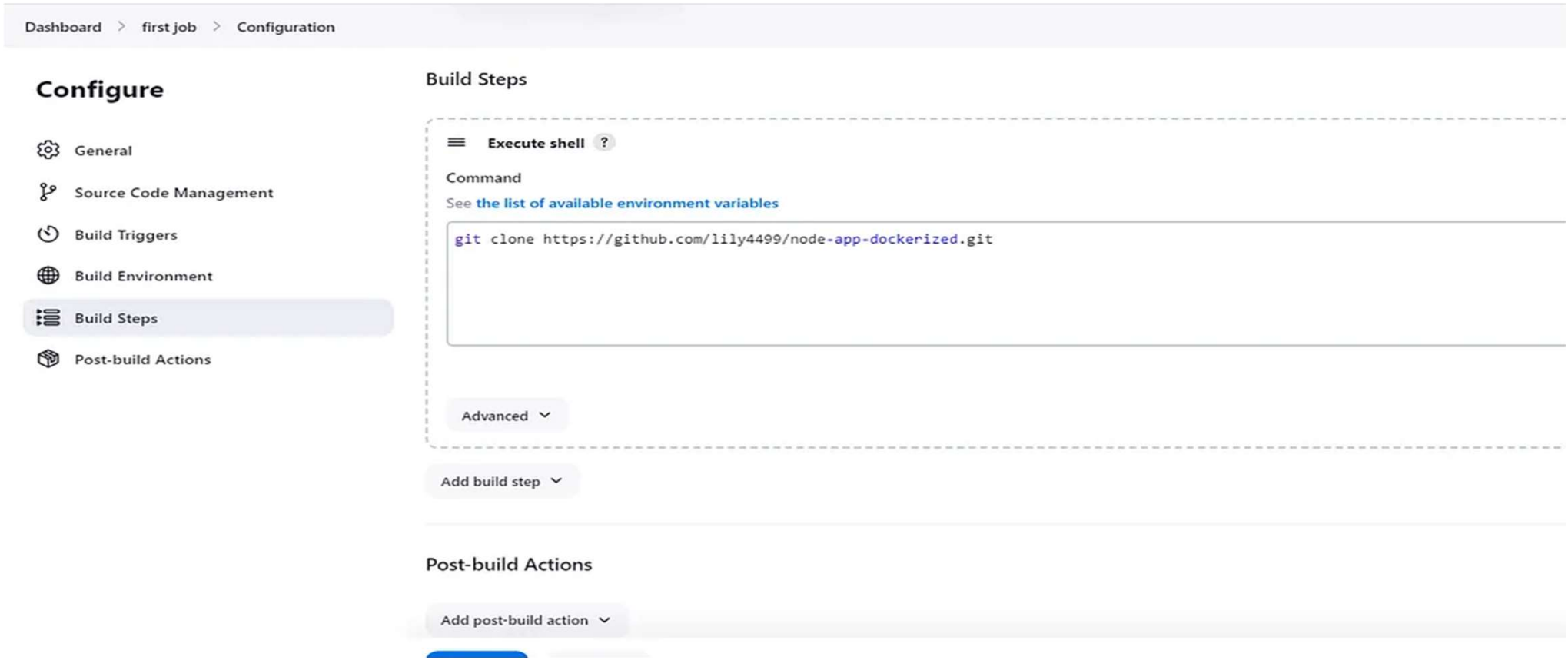
Integrating Source code management and builds with Jenkins

- Step 5: Set Up Build Triggers
- Scroll down to the “Build Triggers” section in the job configuration page.
- Choose the appropriate trigger for your workflow, such as “Poll SCM” to periodically check for changes in the repository or “Build when a change is pushed to GitHub” for GitHub webhook integration.
- Configure the trigger settings as per your requirements.
- Click “Save” to apply the changes.

Integrating Source code management and builds with Jenkins

- Step 6: Define Build Steps
- Scroll down to the “Build” section in the job configuration page.
- Click on “Add build step” and choose the appropriate build step(s) for your project, such as executing shell commands, running Maven or Gradle builds, etc.
- Configure the build steps according to your project requirements.
- Click “Save” to apply the changes.

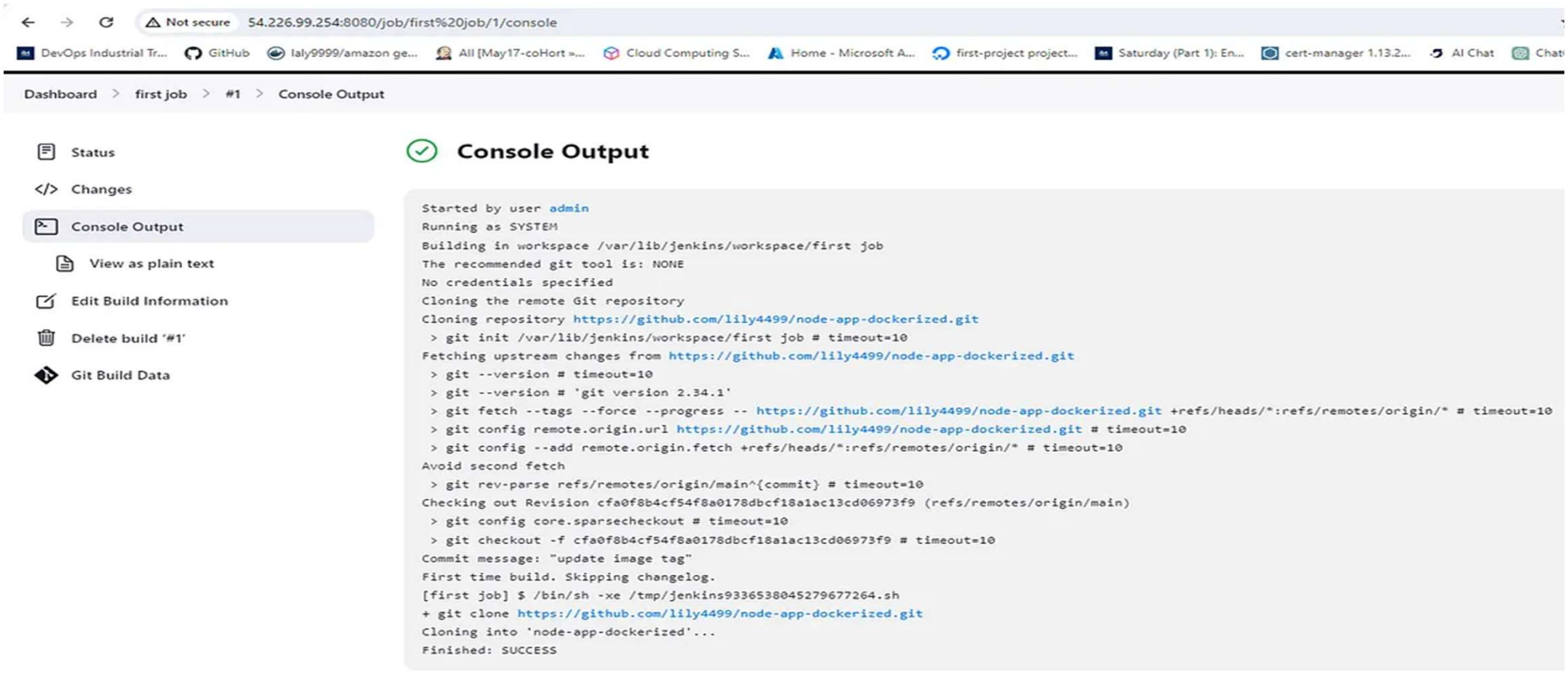
Integrating Source code management and builds with Jenkins



Integrating Source code management and builds with Jenkins

- Step 7: Run the Jenkins Job
- Go back to the Jenkins dashboard.
- Find your newly created job and click on “Build Now” to run the job.
- Jenkins will clone the Git repository, perform the defined build steps, and provide the build status.
- You can view the build console output to monitor the progress and debug any issues.

Integrating Source code management and builds with Jenkins



Integrating Source code management and builds with Jenkins

- Jenkins can accommodate everything from build automation and pipeline management to test automation and monitoring.
- 1. Pipeline Plugin

- Description: The Pipeline Plugin, also known as “Pipeline as Code,” is one of the most powerful plugins in Jenkins.

- It allows you to define complex build pipelines in a simple script format using the Jenkins DSL (Domain Specific Language).

- Pipelines can be defined in a Jenkinsfile, which resides in the project’s source repository.

- Use Case: Ideal for creating multi-step build, test, and deployment workflows.

Integrating Source code management and builds with Jenkins

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'mvn clean install'
      }
    }
    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }
    stage('Deploy') {
      steps {
        sh 'scp target/*.jar user@server:/path/to/deploy'
      }
    }
  }
}
```

Benefit: The Pipeline Plugin supports complex automation sequences, making it essential for implementing CI/CD pipelines.

Integrating Source code management and builds with Jenkins

2. Git Plugin

•**Description**: The Git Plugin integrates Git into Jenkins, allowing Jenkins to pull source code from Git repositories.

•**Use Case**: It's essential for projects using Git for version control, enabling Jenkins to clone, fetch, and manage code changes from Git.

•**Example**:

Integrating Source code management and builds with Jenkins

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        git url: 'https://github.com/user/repository.git', branch: 'main'
      }
    }
    stage('Build') {
      steps {
        sh 'mvn clean install'
      }
    }
  }
}
```

Benefit: Automates source code retrieval and supports integrations with GitHub, GitLab, Bitbucket, etc.

Integrating Source code management and builds with Jenkins

3. Blue Ocean Plugin

•**Description**: Blue Ocean provides an improved user experience, simplifying pipeline visualization and making Jenkins more user-friendly.

•**Use Case**: It's particularly useful for visualizing pipeline stages, especially in complex, multi-branch, and parallelized pipelines.

•**Example**: After installing Blue Ocean, you can access a graphical representation of your pipeline, making it easy to understand each stage's status and monitor the build process.

•**Benefit**: Great for teams needing a clear and interactive visualization of pipeline progress.

Integrating Source code management and builds with Jenkins

- 4. Docker Plugin
- Description: The Docker Plugin allows Jenkins to build and deploy Docker images directly, making it ideal for containerized applications.
- Use Case: Useful for integrating Docker-based workflows where Jenkins can run Docker commands, manage Docker containers, and deploy Docker images.
- Example:
- Benefit: Provides a consistent environment for builds and reduces “works on my machine” issues.

Integrating Source code management and builds with Jenkins

```
pipeline {
  agent {
    docker {
      image 'maven:3-alpine'
      args '-v /root/.m2:/root/.m2'
    }
  }
  stages {
    stage('Build') {
      steps {
        sh 'mvn clean install'
      }
    }
  }
}
```

Integrating Source code management and builds with Jenkins

5. Email Extension Plugin

•**Description**: The Email Extension Plugin provides customized email notifications when builds succeed, fail, or change status.

•**Use Case**: Ensures teams are immediately notified of build status changes, especially useful for quick issue resolution in CI/CD environments.

•**Example**:

•**Benefit**: Configures tailored notifications based on pipeline events, enhancing team communication.

Integrating Source code management and builds with Jenkins

```
post {
  success {
    mail to: 'team@example.com',
        subject: "Build Successful: ${env.JOB_NAME}",
        body: "The build for ${env.JOB_NAME} was
successful!"
  }
  failure {
    mail to: 'team@example.com',
        subject: "Build Failed: ${env.JOB_NAME}",
        body: "The build for ${env.JOB_NAME} failed.
Please check the logs."
  }
 }
```

Integrating Source code management and builds with Jenkins

- 6. Credentials Binding Plugin
- **Description**: This plugin securely injects credentials into build jobs without exposing them in the logs.
- **Use Case**: Essential for securely managing sensitive information, such as API tokens, passwords, or SSH keys.
- **Example**:
- **Benefit**: Enhances collaboration and awareness within teams, especially in distributed setups.

Integrating Source code management and builds with Jenkins

```
post {
  success {
    slackSend(channel: '#build-status', message: "Build successful: ${env.JOB_NAME}")
  }
  failure {
    slackSend(channel: '#build-status', message: "Build failed: ${env.JOB_NAME}")
  }
}
```

Integrating Source code management and builds with Jenkins

- 8. Artifactory Plugin
- **Description**: This plugin integrates Jenkins with Artifactory, allowing Jenkins to deploy artifacts directly to Artifactory repositories.
- **Use Case**: Useful in environments where binary management and artifact storage are essential, such as Maven or npm repositories.
- **Example**:
- **Benefit**: Efficiently manages, version-controls, and reuses artifacts across different projects.

Integrating Source code management and builds with Jenkins

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'mvn clean install'
      }
    }
    stage('Upload to Artifactory') {
      steps {
        script {
          def server = Artifactory.server 'my-artifactory-server'
          def uploadSpec = """{
            "files": [
              {
                "pattern": "target/*.jar",
                "target": "libs-release-local/"
              }
            ]
          }"""
          server.upload(uploadSpec)
        }
      }
    }
  }
}
```

Integrating Source code management and builds with Jenkins

- 9. JUnit Plugin
- **Description**: The JUnit Plugin parses test result reports in JUnit XML format, generating reports that display pass/fail statuses for each test.
- **Use Case**: Essential for monitoring the health and stability of the codebase by providing detailed test result feedback.
- **Example**:
- **Benefit**: Delivers critical test data, helping developers identify and resolve issues early.

Integrating Source code management and builds with Jenkins

```
pipeline {
  agent any
  stages {
    stage('Build and Test') {
      steps {
        sh 'mvn clean test'
      }
    }
  }
  post {
    always {
      junit '**/target/surefire-reports/*.xml'
    }
  }
}
```

Integrating Source code management and builds with Jenkins

10. Role-based Authorization Strategy Plugin

- **Description**: Provides a role-based authorization strategy to control access to projects, views, and other parts of Jenkins.

- **Use Case**: Crucial for teams requiring granular permission control, where different users have different levels of access.

- **Example**: After installing, configure user roles and assign them to different project folders or jobs within Jenkins, ensuring secure access control.

- **Benefit**: Strengthens security by enforcing least privilege and access control policies.

Integrating Source code management and builds with Jenkins

- These plugins offer versatility and enhancements to Jenkins' core functionality, making it easier for DevOps teams to automate, manage, and monitor complex CI/CD workflows.

- Each plugin is designed to solve specific challenges within the CI/CD pipeline, and together, they build a robust environment for rapid development and deployment.

Recording tests and artifacts

- While testing is a critical part of a good continuous delivery pipeline, most people don't want to sift through thousands of lines of console output to find information about failing tests.
- To make this easier, Jenkins can record and aggregate test results so long as your test runner can output test result files.
- Jenkins typically comes bundled with the junit step, but if your test runner cannot output JUnit-style XML reports, there are additional plugins which process practically any widely-used test report format.

Recording tests and artifacts

Jenkinsfile (Declarative Pipeline)

```
pipeline {
    agent any
    stages {
        stage('Test') {
            steps {
                sh './gradlew check'
            }
        }
    }
    post {
        always {
            junit 'build/reports/**/*.xml'
        }
    }
}
```

This will *always* grab the test results and let Jenkins track them, calculate trends and report on them. A Pipeline that has failing tests will be marked as "UNSTABLE", denoted by yellow in the web UI. That is distinct from the "FAILED" state, denoted by red.

Recording tests and artifacts

```
Jenkinsfile (Declarative Pipeline)
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                sh './gradlew build'
            }
        }
        stage('Test') {
            steps {
                sh './gradlew check'
            }
        }
    }

    post {
        always {
            archiveArtifacts artifacts: 'build/libs/**/*.jar', fingerprint: true
            junit 'build/reports/**/*.xml'
        }
    }
}
```

When there are test failures, it is often useful to grab built artifacts from Jenkins for local analysis and investigation. This is made practical by Jenkins's built-in support for storing "artifacts", files generated during the execution of the Pipeline.

Recording tests and artifacts

- If more than one parameter is specified in the archiveArtifacts step, then each parameter's name must explicitly be specified in the step code - i.e. artifacts for the artifact's path and file name and fingerprint to choose this option.

- If you only need to specify the artifacts' path and file name/s, then you can omit the parameter name artifacts - e.g.
  archiveArtifacts 'build/libs/**/*.jar'

- Recording tests and artifacts in Jenkins is useful for quickly and easily surfacing information to various members of the team.

- How to **tell** those members of the team what's been happening in our Pipeline.

Cleaning up and notifications

Jenkinsfile (Declarative Pipeline)

```
pipeline {
    agent any
    stages {
        stage('No-op') {
            steps {
                sh 'ls'
            }
        }
    }
    post {
        always {
            echo 'One way or another, I have finished'
            deleteDir() /* clean up our workspace */
        }
        success {
            echo 'I succeeded!'
        }
        unstable {
            echo 'I am unstable :/'
        }
        failure {
            echo 'I failed :('
        }
        changed {
            echo 'Things were different before...'
        }
    }
}
```

A Pipeline is guaranteed to run at the end of a Pipeline's execution, we can add some notification or other steps to perform finalization, notification, or other end-of-Pipeline tasks.

Cleaning up and notifications

Email

```
post {
    failure {
        mail to: 'team@example.com',
             subject: "Failed Pipeline:
${currentBuild.fullDisplayName}",
             body: "Something is wrong with
${env.BUILD_URL}"
    }
}
```

Slack

```
post {
    success {
        slackSend channel: '#ops-room',
                  color: 'good',
                  message: "The pipeline
${currentBuild.fullDisplayName} completed
successfully."
    }
}
```

There are plenty of ways to send notifications, below are a few snippets demonstrating how to send notifications about a Pipeline to an email or a Slack channel.

Now that the team is being notified when things are failing, unstable, or even succeeding we can finish our continuous delivery pipeline with the exciting part: shipping!

Cleaning up and notifications

- Deployment
- The most basic continuous delivery pipeline will have, at minimum, three stages which should be defined in a Jenkinsfile: Build, Test, and Deploy.

- We will focus primarily on the Deploy stage, but it should be noted that stable Build and Test stages are an important precursor to any deployment activity.

Setting up the Continuous Integration pipeline

- Understanding Jenkins Pipeline

- A **Jenkins Pipeline** is a powerful set of features that allows developers to automate the entire software delivery process — such as building, testing, and deploying — by defining each stage in a structured sequence.

- It enables teams to represent their CI/CD workflows as code, which can be stored in version control systems, making them easy to track, update, and reuse.

- Jenkins Pipelines come in two styles — **Declarative** and **Scripted** — offering flexibility to suit different project needs and coding preferences.

Building a CI/CD Pipeline with Jenkins

Before creating a CI/CD pipeline with Jenkins, make sure to complete the following prerequisites:

1.Install Jenkins

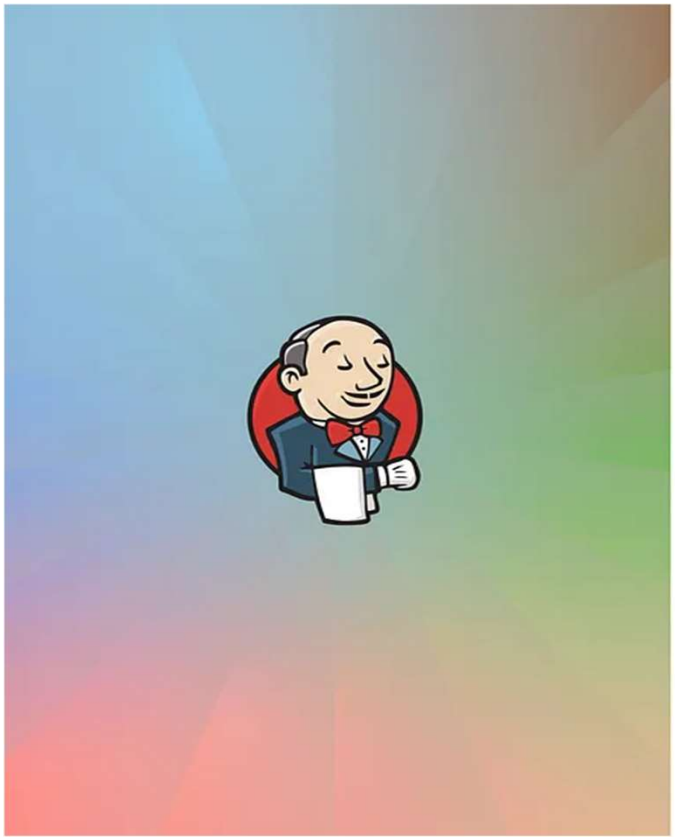
Download and install Jenkins from the official Jenkins website.

2. Alternatively, you can use Docker to install Jenkins by running this command:

```
docker run -d \
  --name jenkins \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts
```

Once installed, Jenkins will be accessible at http://localhost:8080.

Building a CI/CD Pipeline with Jenkins

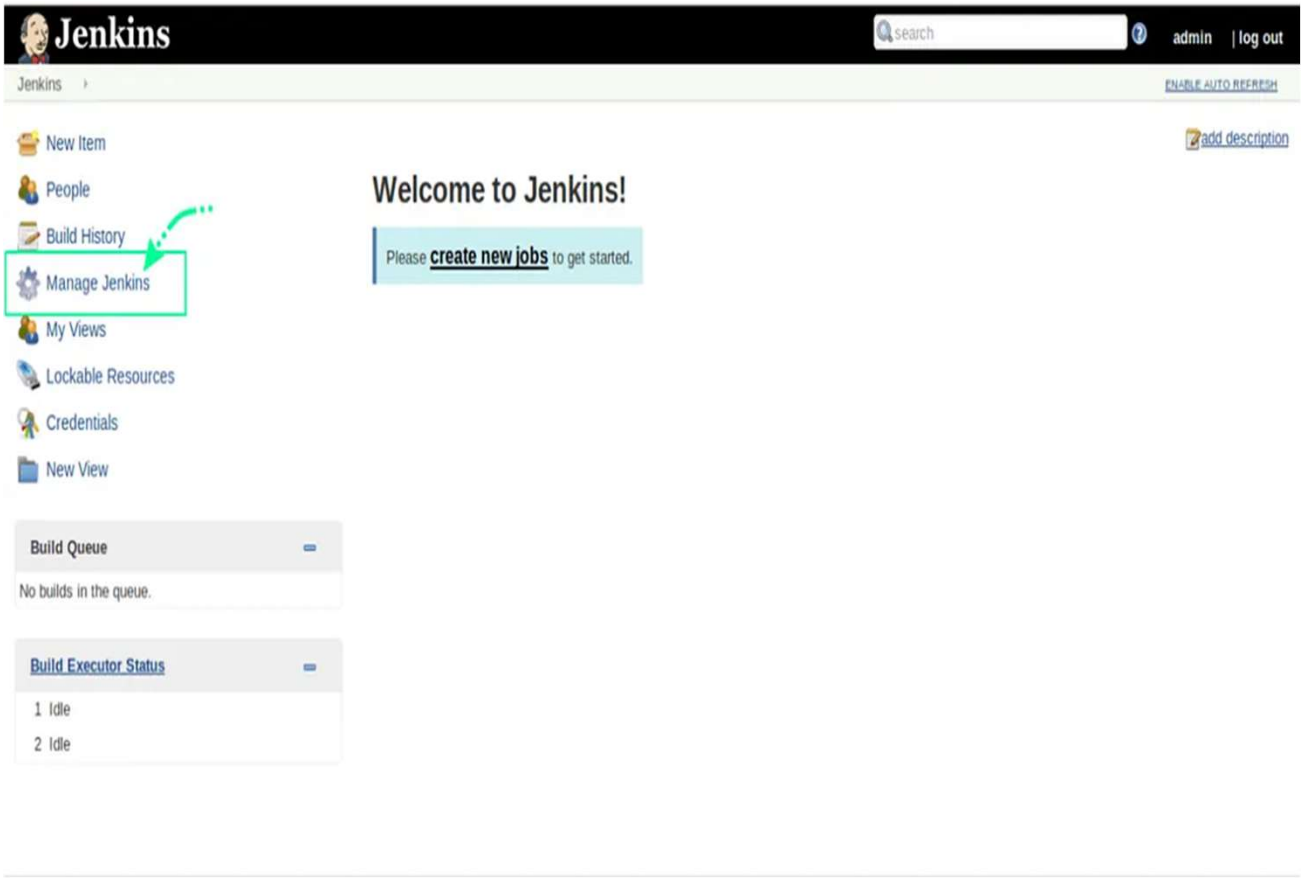


Building a CI/CD Pipeline with Jenkins

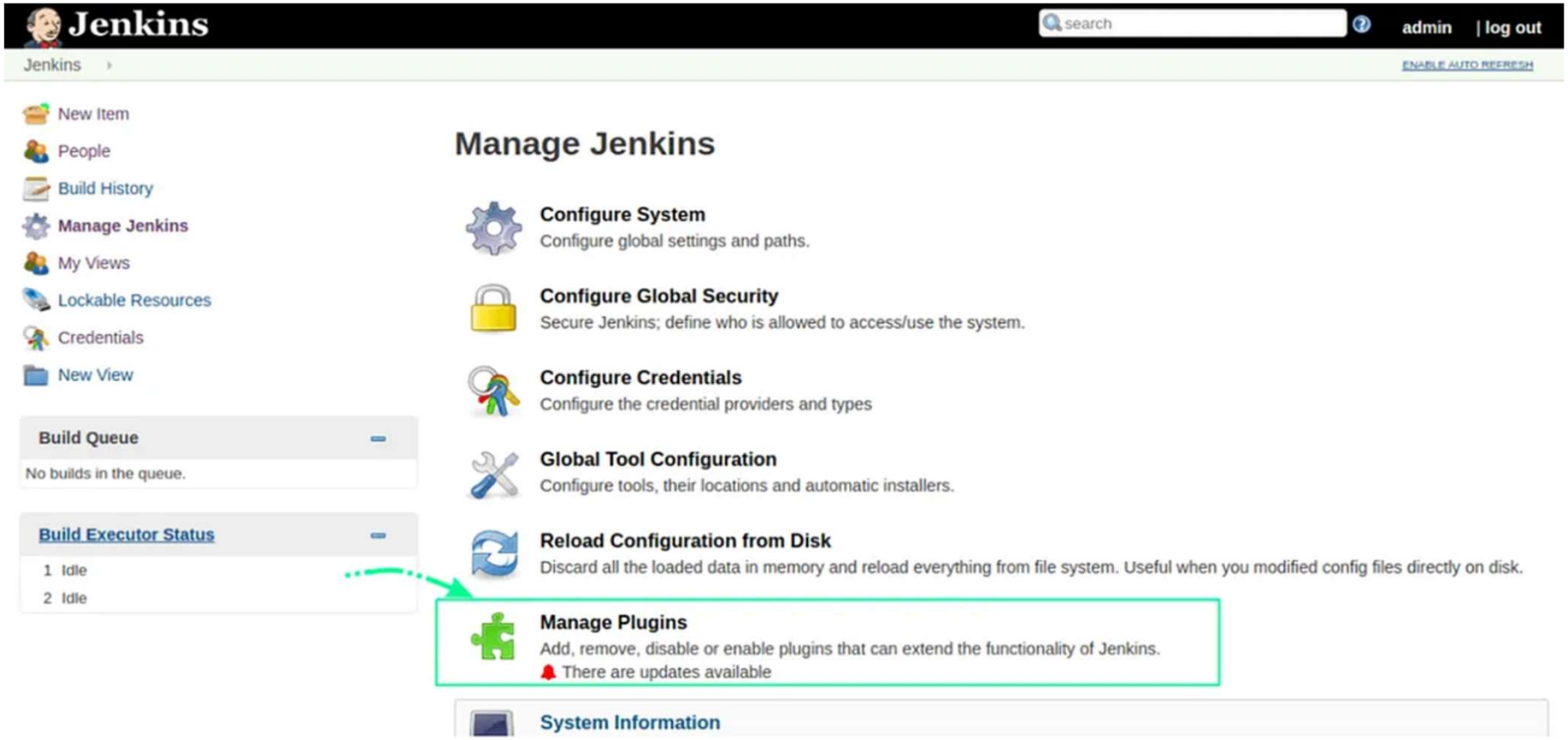
- 2. Configure Jenkins and add necessary plugins

- Setting up Jenkins involves selecting the necessary plugins for the project.

- Commonly used options at the start include the Git plugin and the Pipeline plugin, which support version control and CI/CD processes.



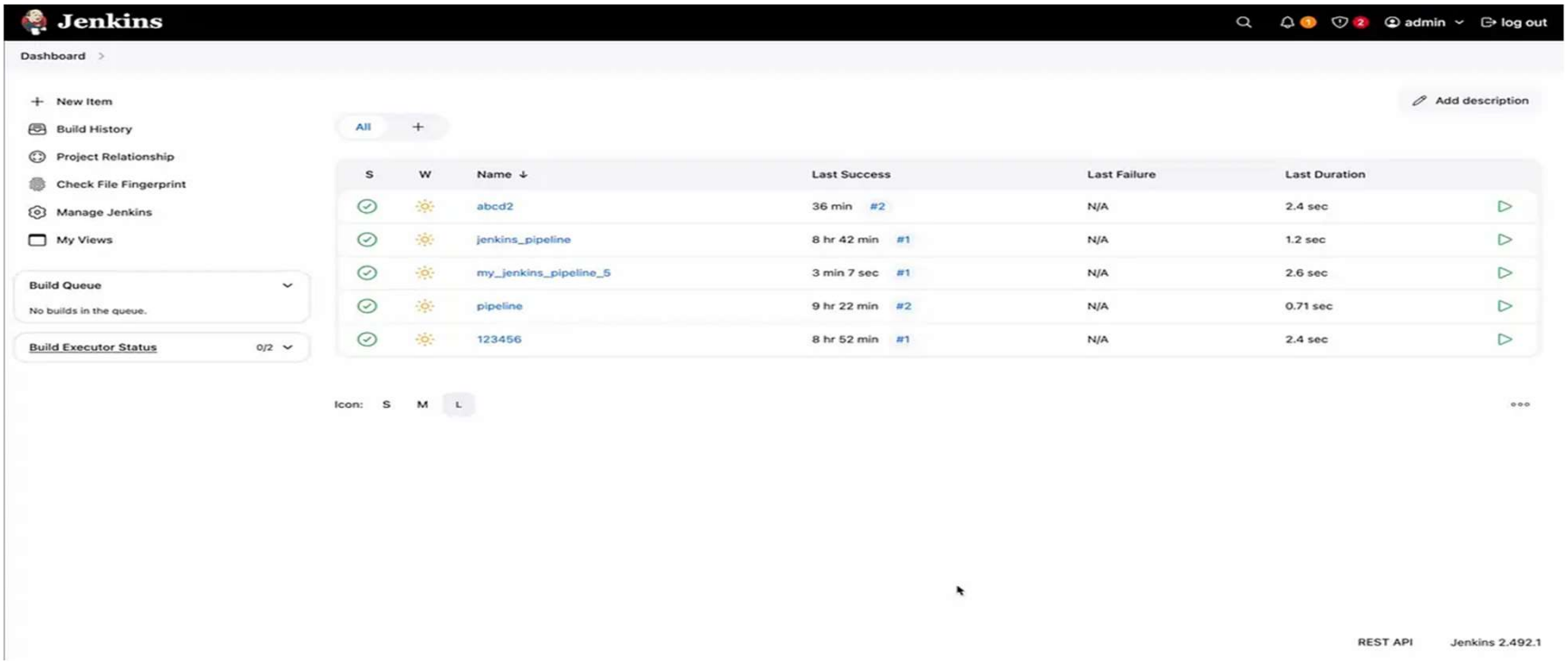
Building a CI/CD Pipeline with Jenkins



Building a CI/CD Pipeline with Jenkins

3. Open Jenkins

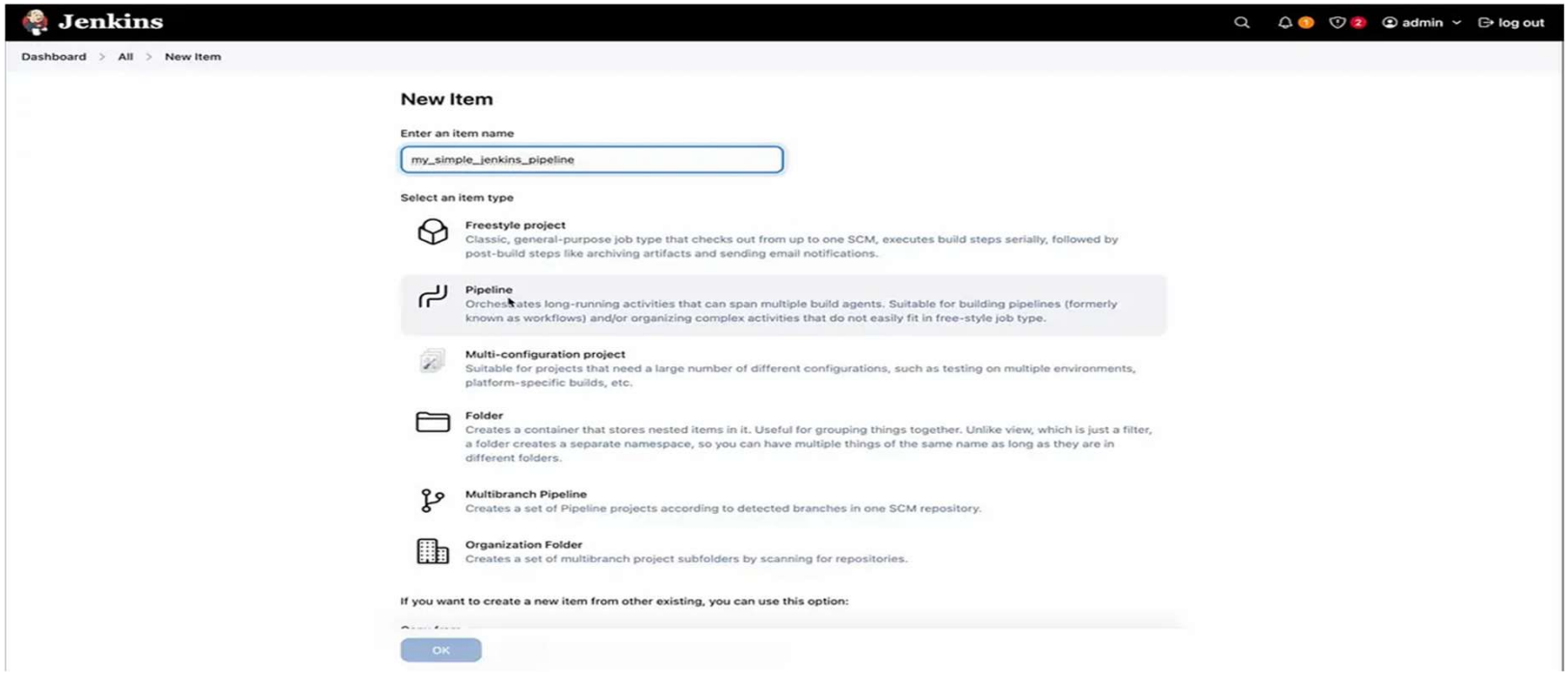
Login to Jenkins and click on “New Item.”



Building a CI/CD Pipeline with Jenkins

4. **Name the pipeline**

Select the “Pipeline” option from the menu, provide a name for the pipeline, and click “OK.”



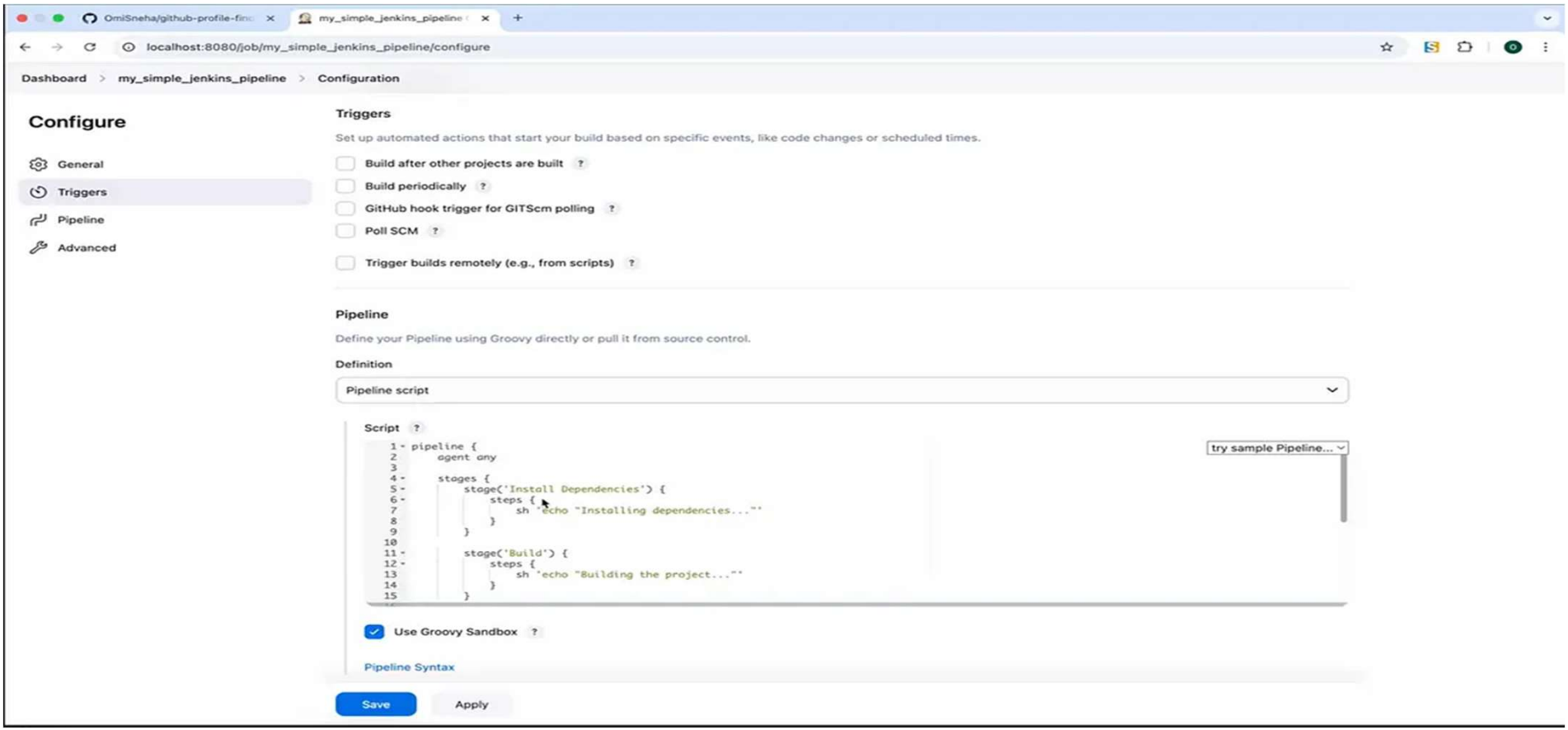
Building a CI/CD Pipeline with Jenkins

- 5. Configure the pipeline
- The CI/CD pipeline can be set up through the pipeline configuration interface, where build triggers and various settings are defined.
- The key part is the ‘Pipeline Definition’ area, where pipeline stages are specified.
- Jenkins allows the use of both declarative and scripted formats for defining these stages.
- Here’s a basic example of a ‘Hello World’ pipeline script:

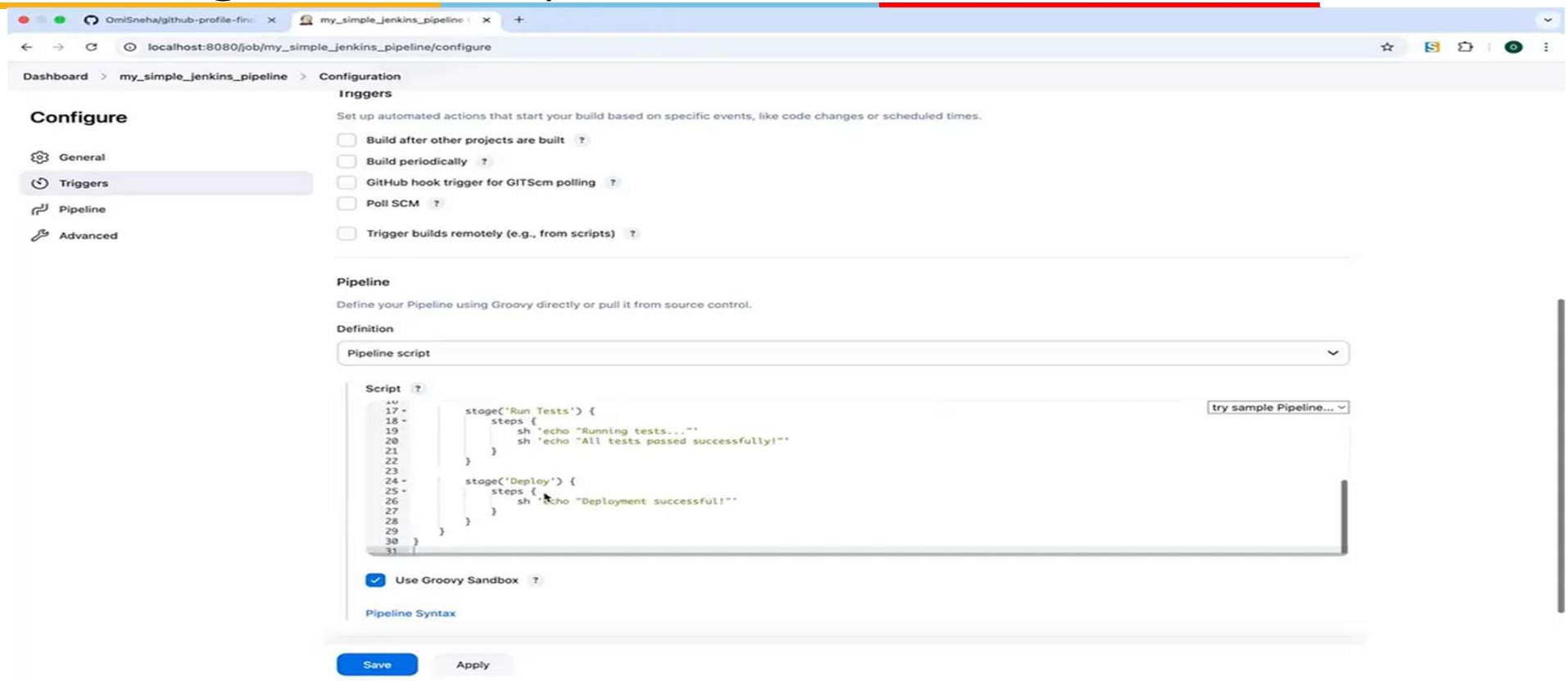
Building a CI/CD Pipeline with Jenkins

```
pipeline {
    agent any

    stages {
        stage('Install Dependencies') {
            steps {
                sh 'echo "Installing dependencies..."'
            }
        }

        stage('Build') {
            steps {
                sh 'echo "Building the project..."'
            }
        }

        stage('Run Tests') {
            steps {
                sh 'echo "Running tests..."'
                sh 'echo "All tests passed successfully!"'
            }
        }

        stage('Deploy') {
            steps {
                sh 'echo "Deployment successful!"'
            }
        }
    }
}
```

Building a CI/CD Pipeline with Jenkins



Building a CI/CD Pipeline with Jenkins



Select Apply and then Save to complete the setup of a basic pipeline.

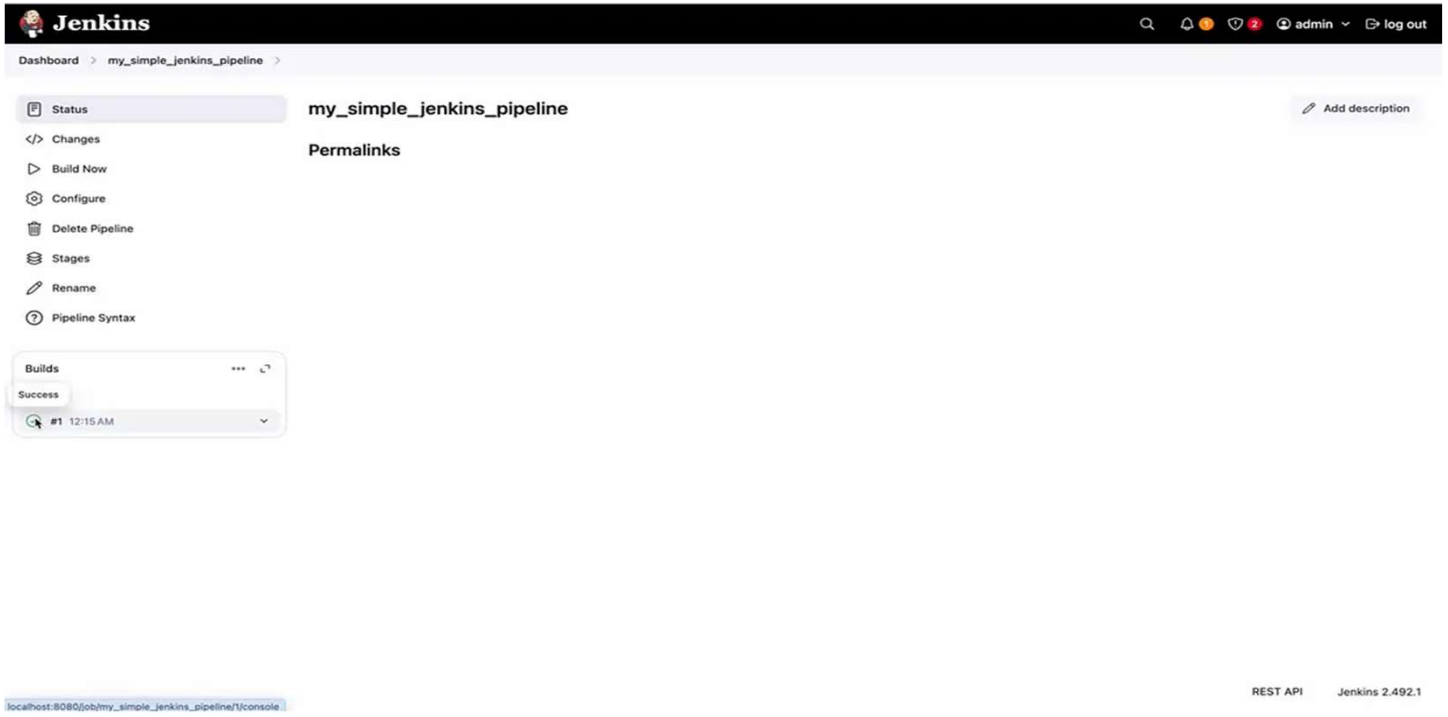
Building a CI/CD Pipeline with Jenkins

6. Trigger the pipeline

Select ‘Build Now’ to initiate the pipeline run.

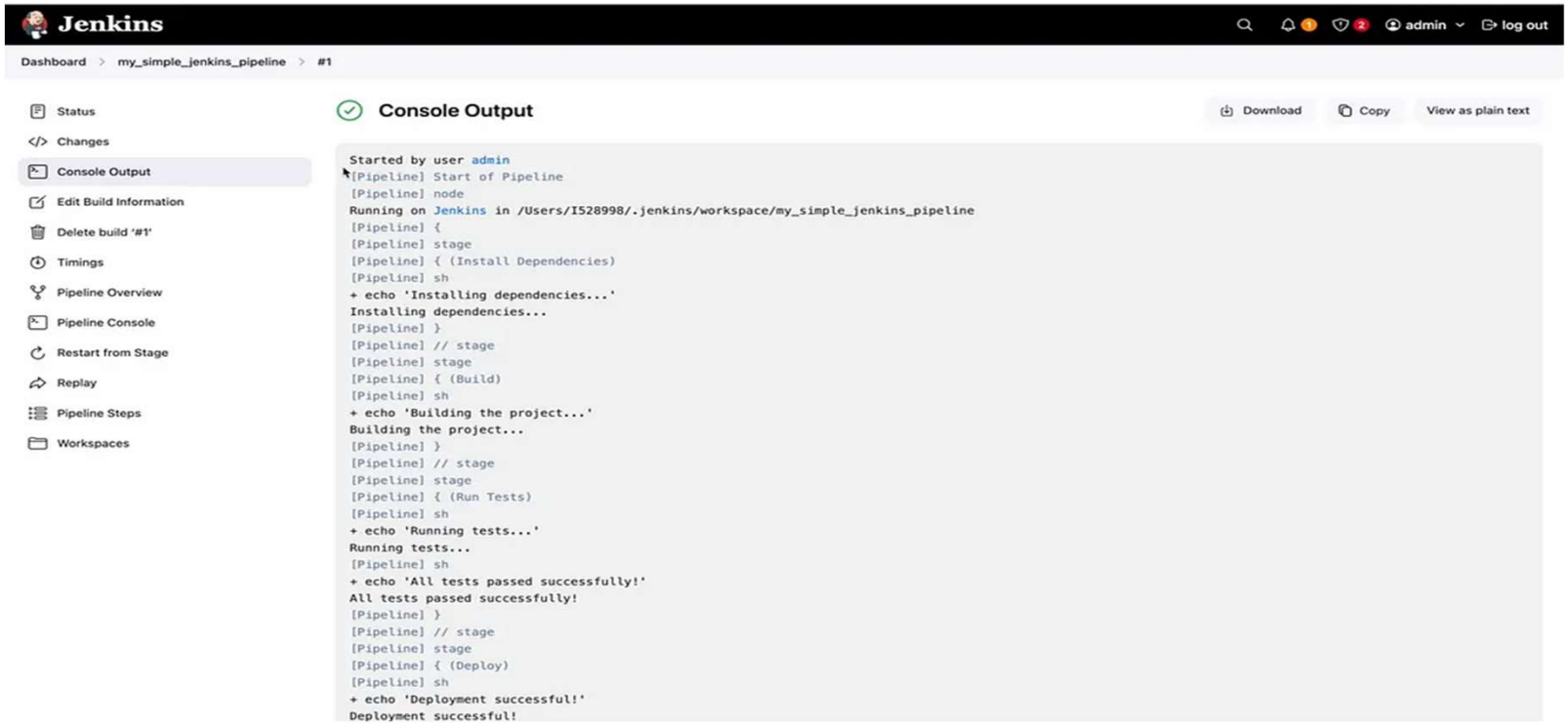
Building a CI/CD Pipeline with Jenkins

This action will trigger the execution of the pipeline stages, with the outcome shown in the ‘Builds’ section. The pipeline is now built successfully:



Building a CI/CD Pipeline with Jenkins

To confirm that the pipeline ran successfully, review the console output of the build process.



The console log reflects a successful build completion status.

Building a CI/CD Pipeline with Jenkins

- 7. Expanding the Pipeline

- The pipeline doesn’t have to stop at basic build and deploy steps. It can be extended to include multiple stages that reflect a real-world workflow — like initial checks, setup, testing, and more granular deployment steps.

- When new stages are added, Jenkins updates the visual stage view accordingly, making it easy to track the execution flow.

- Each stage contributes clarity and structure, which becomes especially helpful in large projects.

Building a CI/CD Pipeline with Jenkins

- 8. Visualizing the Pipeline
- To better understand how the pipeline progresses over time, Jenkins provides visualization tools.

- With the help of plugins like the Pipeline Timeline, it's possible to view a chronological breakdown of all pipeline events.

- This visualization gives teams a clearer picture of execution duration, bottlenecks, and overall efficiency.

- This completes a basic CI/CD pipeline setup using Jenkins, with a flexible foundation that can grow to include version control, test automation, and robust deployment practices.

Integrating Git and Docker with Jenkins

- A key advantage of Jenkins as a CI/CD tool lies in its seamless integration with version control systems such as **Git**.
- Git serves as the foundation for modern source code management, enabling teams to track changes, manage branches, and collaborate effectively.
- When connected to Jenkins, Git repositories become the starting point for automated build and deployment workflows.
- Every time a commit is pushed to a Git repository — hosted on platforms like **GitHub**, **GitLab**, or **Bitbucket** — Jenkins can be configured to automatically detect these changes.

Integrating Git and Docker with Jenkins

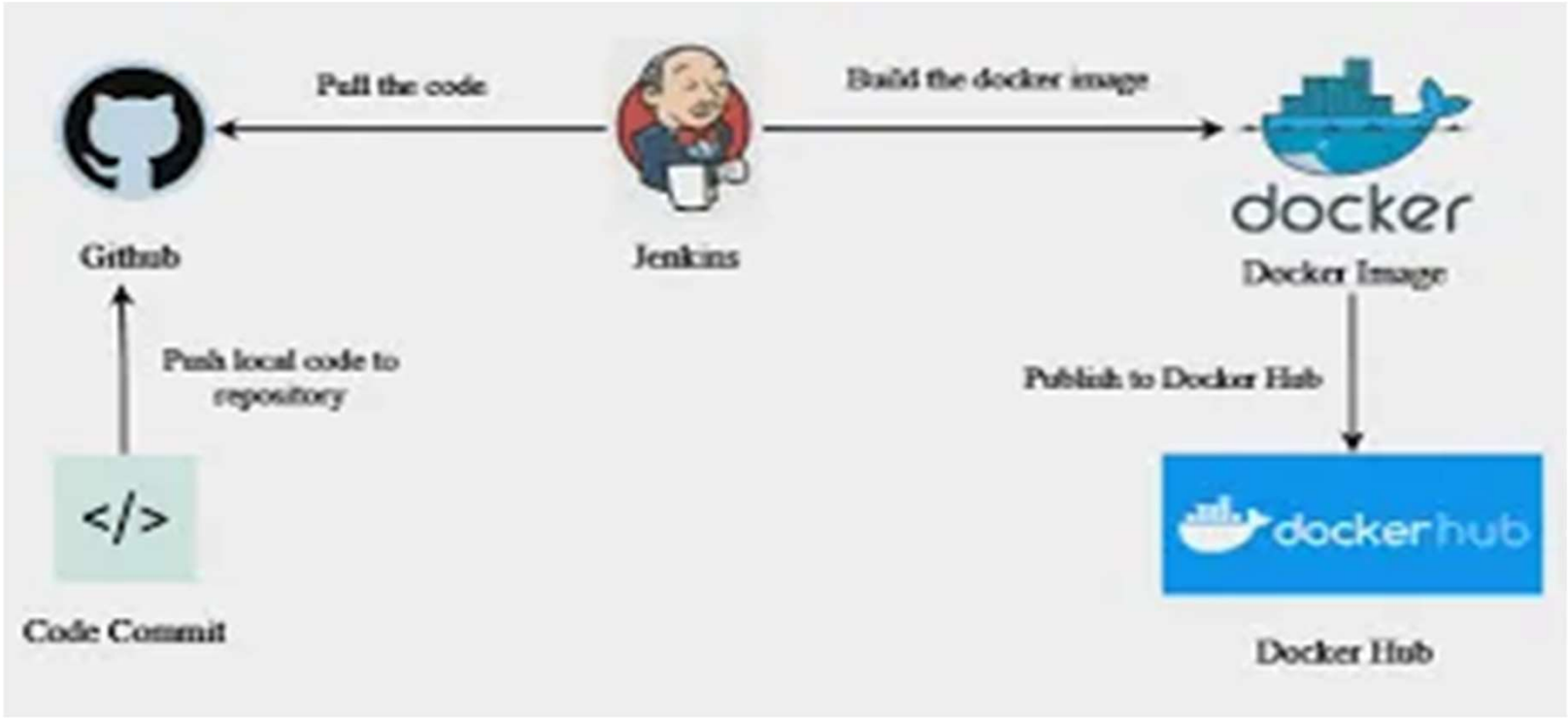
- It then pulls the updated code, initiates a build pipeline, runs tests, and prepares the application for deployment.

- This continuous integration process ensures rapid feedback, code reliability, and reduced risk of integration conflicts.

- Incorporating **Docker** into this workflow further enhances the pipeline's efficiency and consistency.
- Jenkins can build Docker images directly from the version-controlled code, run tests in containerized environments, and push final images to container registries.

- This containerization approach standardizes environments across development, testing, and production, eliminating discrepancies and deployment issues.

Integrating Git and Docker with Jenkins



By combining Git, Jenkins, and Docker, CI/CD pipelines become more robust, scalable, and aligned with modern DevOps practices.

By combining Git, Jenkins, and Docker, CI/CD pipelines become more robust, scalable, and aligned with modern DevOps practices.

Jenkinsfile (Declarative Pipeline)

```
Jenkinsfile (Declarative Pipeline)
pipeline {
    agent any
    options {
        skipStagesAfterUnstable()
    }
    stages {
        stage('Build') {
            steps {
                echo 'Building'
            }
        }
        stage('Test') {
            steps {
                echo 'Testing'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying'
            }
        }
    }
}
```

Stages as Deployment Environments

One common pattern is to extend the number of stages to capture additional deployment environments, like "staging" or "production", as shown in the following snippet.

```
stage('Deploy - Staging') {
    steps {
        sh './deploy staging'
        sh './run-smoke-tests'
    }
}
stage('Deploy - Production') {
    steps {
        sh './deploy production'
    }
}
```

Stages as Deployment Environments

- In this example, we're assuming that whatever "smoke tests" are run by our ./run-smoke-tests script are sufficient to qualify or validate a release to the production environment.
- This kind of pipeline that automatically deploys code all the way through to production can be considered an implementation of "continuous deployment."
- While this is a noble ideal, for many there are good reasons why continuous *deployment* might not be practical, but those can still enjoy the benefits of continuous *delivery*.
- Jenkins Pipeline readily supports both.

Stages as Deployment Environments

- Asking for human input to proceed
- Often when passing between stages, especially environment stages, you may want human input before continuing.
- For example, to judge if the application is in a good enough state to "promote" to the production environment.
- This can be accomplished with the input step.
- In the example below, the "Sanity check" stage actually blocks for input and won't proceed without a person confirming the progress.

Stages as Deployment Environments

Jenkinsfile (Declarative Pipeline)

```
pipeline {
    agent any
    stages {
        /* "Build" and "Test" stages omitted */

        stage('Deploy - Staging') {
            steps {
                sh './deploy staging'
                sh './run-smoke-tests'
            }
        }

        stage('Sanity check') {
            steps {
                input "Does the staging environment look ok?"
            }
        }

        stage('Deploy - Production') {
            steps {
                sh './deploy production'
            }
        }
    }
}
```

Conclusion

We covered the basics of using Jenkins and Jenkins Pipeline. Because Jenkins is extremely extensible, it can be modified and configured to handle practically *any* aspect of automation.

Stages as Deployment Environments

Workflow Overview:

🚀 App → 🛠️ GitHub → 🤖 Jenkins → 🖥️ Dev → 🧪 Staging → 🌐 Production

Key Steps in the Workflow:

1. **App:** User interacts with the application.
2. **GitHub:** Source code is managed in repositories.
3. **Jenkins:** CI/CD pipeline automates builds.
4. **Dev:** Development environment testing.
5. **Staging:** Pre-production validation.
6. **Production:** Live environment for users.

How to implement Self-Healing Pipelines in Jenkins

- Continuous Integration and Continuous Delivery (CI/CD) pipelines are vital for ensuring software development processes are efficient and reliable.
- However, these pipelines often encounter failures due to unforeseen issues like flaky tests, resource exhaustion, or configuration mismatches.
- Self-healing pipelines can autonomously detect, diagnose, and resolve such failures, minimizing downtime and enhancing reliability.

How to implement Self-Healing Pipelines in Jenkins

- What is a Self-Healing Pipeline?
- A self-healing pipeline is a CI/CD pipeline that can automatically detect issues and apply corrective actions without manual intervention.

- These pipelines leverage monitoring tools, machine learning algorithms, and custom scripts to identify failures, determine their root cause, and implement solutions.

- Why Jenkins?
- Jenkins is one of the most popular open-source automation tools for building, testing, and deploying software.

- Its plugin ecosystem and extensibility make it an excellent platform for implementing self-healing pipelines.

Key Features of a Self-Healing Pipeline

- **Failure Detection:** Real-time monitoring of pipeline stages to detect anomalies or failures.
- **Root Cause Analysis (RCA):** Identifying the underlying cause of failures using logs and metrics.
- **Automated Recovery:** Restarting failed steps, allocating additional resources, or applying fixes.
- **Learning Mechanism:** Leveraging historical data to predict and prevent recurring issues.

Implementing a Self-Healing Pipeline in Jenkins

- 1. Setting Up Jenkins for Monitoring
- **Install Monitoring Tools:** Use plugins like the **Build Monitor Plugin** or integrate with external tools like Prometheus and Grafana.

- **Log Aggregation:** Implement centralized logging using the **Logstash** plugin or tools like Elasticsearch.

- **Proactive Monitoring and Alerts**
- Modern DevOps teams leverage tools like **Prometheus**, **Grafana**, and **ELK Stack** to monitor system health in real-time.

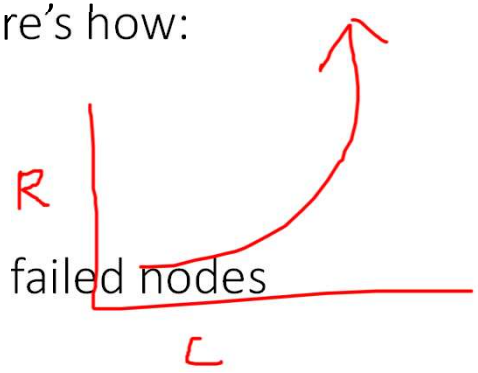
- These tools detect anomalies like performance dips or server issues and send out alerts before the situation escalates.
- *Example: Detecting an overloaded database and notifying engineers (or triggering auto-scaling).*

Implementing a Self-Healing Pipeline in Jenkins

- 2. Automating Failure Detection
- **Build Status Tracking:** Use the **Build Failure Analyzer Plugin** to identify common patterns in build logs.
- **Alerting Mechanisms:** Configure email notifications or integrations with Slack and Microsoft Teams for real-time alerts.

- Automated Remediation and Recovery
- Why wait for a manual fix when your systems can do it themselves? Here's how:

- **Auto-scaling**: Add resources automatically when traffic spikes.



- **Self-healing scripts**: Tools like **Ansible** or **Chef** restart services or replace failed nodes automatically.
- **Kubernetes Rollbacks**: Automatically revert to a stable application version if something breaks.

Implementing a Self-Healing Pipeline in Jenkins

- 3. Root Cause Analysis
- **Error Categorization:** Analyze log files using pattern recognition.
- **Machine Learning Models:** Integrate with AIOps tools to perform RCA using historical data.

- 4. Implementing Recovery Mechanisms
- **Retry Policies:** Configure Jenkins to automatically retry failed steps with backoff intervals.
- **Resource Scaling:** Use Jenkins pipeline scripts to dynamically allocate more resources.
- **Reverting Changes:** Automate rollback of faulty deployments using Git or versioned artifacts.

- Infrastructure as Code (IaC)
- DevOps uses tools like **Terraform** and **CloudFormation** to define infrastructure in code.
- This allows systems to rebuild themselves automatically in case of failure, ensuring consistent and reliable recovery.

Implementing a Self-Healing Pipeline in Jenkins

- 5. Enhancing the Pipeline with Machine Learning
- Use ML models to predict potential failures by analyzing pipeline metrics.
- Integrate AIOps tools like Dynatrace or Splunk to make data-driven decisions.
- 6. Continuous Improvement
- Collect metrics on pipeline performance and failure rates.
- Update scripts and models based on new failure patterns.

Implementing a Self-Healing Pipeline in Jenkins

- CI/CD Pipelines with Fail-Safe Features
- Continuous Integration and Continuous Deployment (CI/CD) pipelines are central to DevOps. With automated testing and rollbacks, your systems can:
- Prevent bad code from going live.
- Revert to the previous version if an issue is detected post-deployment.

Implementing a Self-Healing Pipeline in Jenkins

- Cloud-Native Self-Healing
- In the cloud, self-healing gets even smarter:
- AWS EC2 auto-healing replaces failed instances.
- Kubernetes recreates unhealthy pods automatically.

Implementing a Self-Healing Pipeline in Jenkins

- Building Resilient Systems
- DevOps champions fault tolerance and resiliency:
- **Distributed systems**: Spread your application across regions or zones for redundancy.
- **Graceful degradation**: Ensure core functionality remains intact even if some features fail.

Implementing a Self-Healing Pipeline in Jenkins

- Why It Matters
- Self-healing systems not only reduce downtime but also free up engineering teams to focus on innovation rather than firefighting.
- DevOps practices make these systems robust, scalable, and future-proof.
- As businesses scale and customer expectations soar, self-healing systems powered by DevOps are no longer a luxury — they're a necessity.

Sample Jenkinsfile for a Self-Healing Pipeline

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                script {
                    try {
                        sh 'mvn clean install'
                    } catch (Exception e) {
                        echo "Build failed. Retrying..."
                        retry(2) {
                            sh 'mvn clean install'
                        }
                    }
                }
            }
        }
        stage('Test') {
            steps {
                script {
                    try {
                        sh 'mvn test'
                    } catch (Exception e) {
                        echo "Tests failed. Checking logs..."
                        archiveArtifacts artifacts: '**/target/surefire-reports/*.xml', allowEmptyArchive: true
                    }
                }
            }
        }
```

Sample Jenkinsfile for a Self-Healing Pipeline

```
    stage('Deploy') {
                steps {
                    script {
                        echo "Deploying application..."
                        // Add deployment scripts here
                    }
                }
            }
        }
        post {
            always {
                echo "Cleaning up workspace..."
                cleanWs()
            }
        }
    }
```

Tools and Plugins for Self-Healing Pipelines

- **Build Failure Analyzer Plugin:** Identifies failure patterns in builds.
- **Jenkins Retry Plugin:** Adds retry functionality for steps.
- **Pipeline Utility Steps Plugin:** Enhances pipeline script capabilities.
- **External Integrations:** Tools like Splunk, Prometheus, and Datadog.

Benefits of Self-Healing Pipelines

- **Increased Uptime:** Minimized disruptions due to autonomous failure recovery.
- **Enhanced Productivity:** Developers spend less time troubleshooting pipeline issues.
- **Cost Savings:** Reduced need for manual intervention and faster delivery cycles.
- **Improved Reliability:** Proactively addresses potential failures before they impact the pipeline.

Challenges in Implementing Self-Healing Pipelines

- **Initial Setup:** Requires expertise in Jenkins and automation tools.
- **Complexity:** Debugging automated recovery scripts can be challenging.
- **Tool Integration:** Ensuring seamless integration with monitoring and logging tools.

Future of Self-Healing Pipelines

- With advancements in AI and machine learning, self-healing pipelines are becoming increasingly sophisticated. Future innovations may include:
- Predictive maintenance using real-time analytics.
- Autonomous decision-making for complex failure scenarios.
- Seamless integration with DevSecOps to address security vulnerabilities.

Conclusion

- Implementing a self-healing pipeline in Jenkins is a step towards achieving resilient and reliable CI/CD workflows.

- By automating failure detection, root cause analysis, and recovery, organizations can significantly enhance their software delivery processes.

QUESTIONS AND DISCUSSION
THANK YOU